

Project 1.2: Xây dựng chatbot hỏi đáp tài liệu học tập

Nguyễn Quốc Thái

Nguyễn Phúc Nguyên

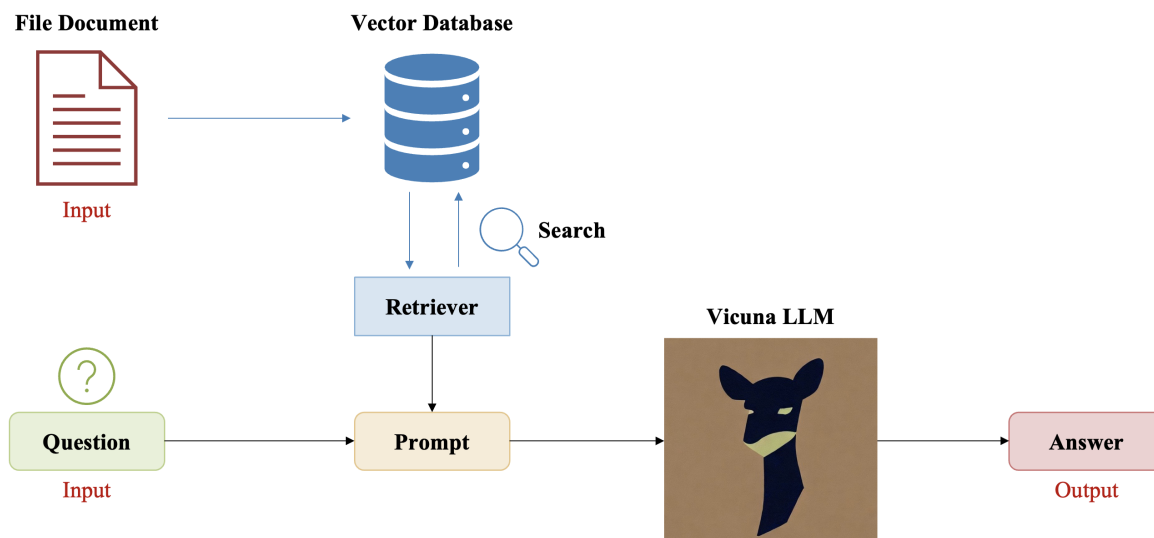
Đinh Quang Vinh

I. Giới thiệu

Hãy tưởng tượng chúng ta có một file PDF dài 50 trang về một chủ đề phức tạp. Chúng ta muốn tìm nhanh câu trả lời cho một câu hỏi cụ thể mà không cần đọc hết toàn bộ tài liệu. Đây chính là bài toán mà chúng ta sẽ giải quyết trong bài viết này.

Large Language Models (LLMs) (Các mô hình ngôn ngữ lớn) là một dạng mô hình Trí tuệ nhân tạo (AI) tiên tiến có khả năng hiểu và tạo ra văn bản giống như con người. Những ứng dụng quen thuộc như ChatGPT hay Gemini đều hoạt động dựa trên phương pháp này. Tuy nhiên, LLM có một hạn chế lớn: chúng chỉ biết những gì đã được huấn luyện từ trước, không thể trả lời các thông tin về tài liệu riêng của chúng ta.

Tổng quan, pipeline (luồng xử lý) của chương trình như sau:



Hình 1: Tổng quan pipeline RAG, từ file PDF đến câu trả lời

Retrieval Augmented Generation (RAG) là một kỹ thuật giúp khắc phục hạn chế này. Ý tưởng rất đơn giản: trước khi để LLM trả lời câu hỏi, chúng ta tìm những đoạn văn bản liên

quan trọng tài liệu, rồi đưa các đoạn đó kèm theo câu hỏi cho LLM. Nhờ vậy, LLM có thể trả lời chính xác dựa trên nội dung tài liệu.

Trong bài viết này, chúng ta sẽ xây dựng một ứng dụng chatbot cho phép:

- **Input:** File tài liệu PDF cần hỏi đáp và một câu hỏi liên quan đến nội dung tài liệu
- **Output:** Câu trả lời dựa trên nội dung tài liệu

Mỗi khối trong pipeline có một nhiệm vụ cụ thể:

- **Chunking:** Chia file PDF thành nhiều đoạn văn bản ngắn. Việc này giúp quá trình phân tích ý nghĩa (embedding) và tìm kiếm chính xác hơn, đồng thời đảm bảo các đoạn nội dung khi ghép lại không vượt quá giới hạn cửa sổ ngữ cảnh (context window) của LLM.
- **Embedding:** Biến mỗi đoạn văn bản thành một dãy số (vector) để máy tính có thể so sánh mức độ giống nhau giữa các đoạn.
- **Lưu vào Database (Vector Database):** Lưu tất cả các vector vào một "kho dữ liệu" để tìm kiếm nhanh.
- **Tìm đoạn liên quan (Retriever):** Khi có câu hỏi, tìm những đoạn văn bản giống nhất với câu hỏi trong kho.
- **Ghép câu hỏi + context (Prompt):** Gộp câu hỏi và các đoạn văn bản tìm được thành một đoạn text hoàn chỉnh.
- **LLM:** Đọc đoạn text đã gộp và sinh ra câu trả lời.

Mục lục

I.	Giới thiệu	1
II.	Chuẩn bị môi trường	4
II.1.	Cài đặt Ollama	4
II.2.	Cài đặt thư viện Python	5
II.3.	Import thư viện	5
III.	Xây dựng chương trình RAG	6
III.1.	Đọc file PDF	6
III.2.	Chunking	7
III.3.	Embedding và lưu vào Vector Database	8
III.4.	Tìm kiếm đoạn liên quan (Retrieve)	9
III.5.	Hỏi đáp với LLM (RAG)	10
IV.	Xây dựng giao diện với Streamlit	11
IV.1.	Cài đặt Streamlit	11
IV.2.	Tạo file ứng dụng	11
IV.3.	Chạy ứng dụng	15
V.	Câu hỏi trắc nghiệm	18
	Phụ lục	22

II. Chuẩn bị môi trường

Trước khi bắt tay vào code, chúng ta cần cài đặt **Ollama**, một công cụ giúp chạy các mô hình AI ngay trên máy tính cá nhân mà không cần GPU đắt tiền hay kết nối internet liên tục.

Ollama là gì? 🤖

Ollama là một công cụ mã nguồn mở giúp tải và chạy các mô hình ngôn ngữ lớn (LLM) trên máy tính cá nhân. Thay vì phải viết hàng chục dòng code để tải model, cấu hình GPU, nén model, Ollama gói gọn tất cả vào 2 lệnh: `ollama pull` (tải model) và `ollama.chat()` (hỏi đáp). Ollama hoạt động như một "server nhỏ" chạy ngầm trên máy, các chương trình Python gọi vào qua API.

II.1. Cài đặt Ollama

Trên máy cá nhân (Windows/macOS/Linux):

Truy cập ollama.com và tải bản cài đặt phù hợp với hệ điều hành. Sau khi cài xong, mở terminal và chạy:

Tải model qua Ollama (máy cá nhân)

```
1
2
3 # Tải mô hình LLM (sinh văn bản)
4 ollama pull vicuna:7b-v1.5-q5_1
5
6 # Tải mô hình Embedding (chuyển text thành vector)
7 ollama pull bge-m3
```

Trên Google Colab:

Colab không có Ollama sẵn, chúng ta cần cài và khởi chạy thủ công:

Cài đặt và khởi chạy Ollama trên Colab

```
1 # Cài đặt các thư viện hệ thống cần thiết
2 !sudo apt-get update && sudo apt-get install -y zstd pciutils lshw
3
4 # Cài đặt Ollama trên Colab
5 !curl -fsSL https://ollama.com/install.sh | sh
6
7 # Tắt Flash Attention để tránh lỗi tương thích phần cứng trên Colab
8 import os, subprocess, time
9 os.environ['OLLAMA_FLASH_ATTENTION'] = 'false'
10
11 # Khởi chạy Ollama server ở chế độ nền
12 subprocess.Popen(["ollama", "serve"]); time.sleep(30)
13
14 # Tải 2 model cần dùng
15 !ollama pull vicuna:7b-v1.5-q5_1
```

```
16 !ollama pull bge-m3
```

Trong đoạn code trên:

- `apt-get install...`: Cài đặt các thư viện hệ thống hỗ trợ Ollama chạy tốt trên Ubuntu của Colab.
- `curl -fsSL ... | sh`: Tải và cài đặt Ollama tự động.
- `os.environ['OLLAMA_FLASH_ATTENTION'] = 'false'`: Tắt tính năng Flash Attention để tránh lỗi tương thích hoặc crash phần cứng trên một số loại GPU của Colab.
- `subprocess.Popen(["ollama", "serve"])`: Khởi chạy Ollama server chạy ngầm. Ollama cần chạy như một server để nhận các yêu cầu từ Python.
- `time.sleep(30)`: Đợi 30 giây cho server khởi động xong trước khi tiếp tục.
- `ollama pull vicuna:7b-v1.5-q5_1`: Tải mô hình **Vicuna-7B**, một LLM mã nguồn mở được tinh chỉnh từ LLaMA; mô hình mạnh ở tiếng Anh, khả năng tiếng Việt ở mức cơ bản.
- `ollama pull bge-m3`: Tải mô hình **BGE-M3** dùng cho embedding, hỗ trợ đa ngôn ngữ bao gồm tiếng Việt.

Thời gian tải model

Lần đầu chạy `ollama pull`, Ollama sẽ tải model về máy (khoảng 4-5GB cho mỗi model). Quá trình này chỉ cần thực hiện một lần duy nhất. Các lần chạy sau, Ollama dùng lại model đã tải mà không cần tải lại.

II.2. Cài đặt thư viện Python

Cài đặt thư viện Python

```
1 !pip install -q pypdf chromadb ollama
```

Chỉ cần 3 thư viện:

- **pypdf**: Đọc nội dung file PDF.
- **chromadb**: Lưu trữ và tìm kiếm vector (vector database).
- **ollama**: Giao tiếp với Ollama server để tạo embedding và sinh văn bản.

II.3. Import thư viện

Import thư viện

```
1 import pypdf
2 import chromadb
3 import ollama
```

So với cách dùng LangChain (cần import 10+ module từ nhiều gói khác nhau), ở đây chúng ta chỉ cần 3 dòng import. Mỗi thư viện đảm nhận một vai trò rõ ràng.

III. Xây dựng chương trình RAG

Với các thư viện đã sẵn sàng, chúng ta bắt đầu xây dựng pipeline RAG theo 5 bước: Đọc PDF → Cắt nhỏ → Embedding + Lưu vector → Tìm kiếm → Hỏi đáp.

III.1. Đọc file PDF

Bước đầu tiên là đọc toàn bộ nội dung text từ file PDF. Chúng ta sử dụng thư viện pypdf để thực hiện:

Đọc nội dung file PDF

```
1 # Đọc file PDF
2 reader = pypdf.PdfReader("./YOLOv10_Tutorials.pdf")
3
4 # Ghép nội dung tất cả các trang thành 1 chuỗi text
5 full_text = "\n".join(page.extract_text() or "" for page in reader.pages)
6
7 print("Số trang:", len(reader.pages))
8 print("Tổng ký tự:", len(full_text))
```

Trong đoạn code trên:

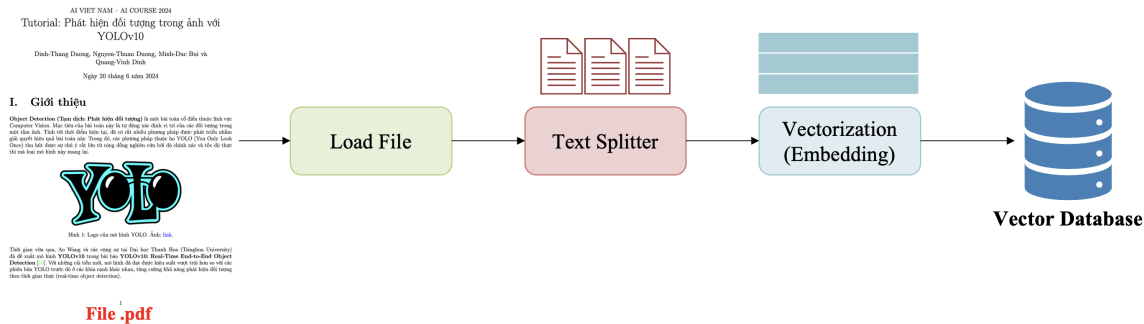
- `PdfReader("./YOLOv10_Tutorials.pdf")`: Mở file PDF và tạo đối tượng reader. Các bạn thay đường dẫn bằng file PDF của mình.
- `page.extract_text()`: Trích xuất text từ mỗi trang. `or ""` xử lý trường hợp trang không có text (ví dụ: trang chỉ chứa hình ảnh).
- `'\n'.join(...)`: Ghép nội dung tất cả các trang lại, mỗi trang cách nhau bởi 1 dòng trống.

File PDF mẫu

File `YOLOv10_Tutorials.pdf` là tài liệu mẫu dùng trong bài. Các bạn có thể thay bằng bất kỳ file PDF nào (slide bài giảng, bài nghiên cứu, tài liệu hướng dẫn). Trên Google Colab, hãy upload file PDF lên hoặc dùng lệnh `!gdown` để tải từ Google Drive.

III.2. Chunking

File PDF thường chứa lượng thông tin rất lớn. Nếu đưa toàn bộ file vào LLM cùng lúc, văn bản có thể vượt quá giới hạn *context window* của model. Hơn nữa, quá nhiều nội dung không liên quan sẽ làm giảm độ chính xác của câu trả lời.



Hình 2: Minh họa quy trình cắt nhỏ văn bản thành các chunk

Vì vậy, chúng ta cần cắt nhỏ văn bản thành các đoạn ngắn gọi là **chunk**. Trong bài này, chúng ta tự viết hàm chunking bằng Python thuần:

Hàm cắt nhỏ văn bản (chunking)

```
1 def chunk_text(text, size=1000, overlap=200):
2     """Cắt text thành các đoạn nhỏ có độ dài tối đa 'size' ký tự,
3     với 'overlap' ký tự trùng lặp giữa 2 đoạn liên tiếp."""
4     paras = [p.strip() for p in text.split("\n") if p.strip()]
5     chunks, cur = [], ""
6     for p in paras:
7         if len(cur) + len(p) + 1 <= size:
8             cur += p + "\n"
9         else:
10            if cur:
11                chunks.append(cur.strip())
12                cur = (cur[-overlap:] + p + "\n") if overlap else (p + "\n")
13            if cur.strip():
14                chunks.append(cur.strip())
15    return chunks
16
17 chunks = chunk_text(full_text)
18 print("Số chunks:", len(chunks))
19 print(chunks[0][:300]) # Xem 300 ký tự đầu của chunk đầu tiên
```

Hàm `chunk_text` hoạt động như sau:

- **Tách đoạn văn:** `text.split('\n')` tách text thành các đoạn theo dấu xuống dòng, loại bỏ dòng trống.

- **Gộp từng đoạn:** Duyệt qua từng đoạn, gộp liên tiếp cho tới khi tổng chiều dài vượt quá size (1000 ký tự). Khi vượt quá, chunk hiện tại được lưu lại và bắt đầu chunk mới.
- **Overlap:** cur[-overlap:] lấy 200 ký tự cuối của chunk trước làm phần mở đầu cho chunk tiếp theo. Mục đích là giữ ngữ cảnh liên tục, tránh mất thông tin ở ranh giới giữa 2 chunk.

Tại sao cần Overlap? ❗

Hãy hình dung khi đọc sách và đánh dấu trang. Nếu một câu quan trọng nằm đúng ranh giới giữa 2 trang, cắt mà không overlap sẽ chia đôi câu đó. Overlap giống như "vùng đệm": vài dòng cuối trang cũ được lặp lại ở đầu trang mới, đảm bảo không câu nào bị cắt đôi.

💡 Chọn chunk_size và chunk_overlap 💡

chunk_size quá nhỏ (ví dụ: 200) sẽ làm mất ngữ cảnh, câu trả lời thiếu thông tin. chunk_size quá lớn (ví dụ: 5000) sẽ lẫn nhiều thông tin nhiễu. Giá trị 1000 là một điểm khởi đầu tốt cho hầu hết các trường hợp. chunk_overlap nên bằng 10-20% của chunk_size.

III.3. Embedding và lưu vào Vector Database

Sau khi cắt nhỏ văn bản, chúng ta cần chuyển mỗi đoạn text thành một dãy số (gọi là **vector**) rồi lưu vào **vector database**. Bước này gồm 2 phần: tạo embedding và lưu trữ.

Embedding là gì? ❗

Embedding là quá trình chuyển văn bản thành một dãy số (vector) sao cho các đoạn văn có ý nghĩa giống nhau sẽ có vector gần nhau trong không gian. Ví dụ: câu "YOLO là mô hình phát hiện vật thể" và "Object Detection với YOLO" sẽ có vector rất gần nhau, dù dùng từ ngữ khác nhau. Hãy hình dung đơn giản: nếu mỗi đoạn văn bản là một điểm trên bản đồ, thì embedding giúp đặt các đoạn có nội dung giống nhau gần nhau trên bản đồ đó.

Tạo embedding và lưu Vector Database

```

1 # Hàm tạo embedding từ danh sách text
2 def embed(texts):
3     """Chuyển danh sách chuỗi text thành danh sách vector."""
4     return ollama.embed(model="bge-m3", input=texts)["embeddings"]
5
6 # Tạo vector database trong bộ nhớ
7 client = chromadb.Client()
8 collection = client.get_or_create_collection("rag")
9
10 # Thêm tất cả chunks vào database
11 collection.add(
12     ids=[str(i) for i in range(len(chunks))], # ID duy nhất cho mỗi chunk
13     documents=chunks,                          # Nội dung text gốc
14     embeddings=embed(chunks),                  # Vector tương ứng

```

```

15 )
16 print("Đã index:", collection.count(), "chunks")

```

Trong đoạn code trên:

- `ollama.embed(model="bge-m3", input=texts)`: Gọi Ollama để chuyển text thành vector. Model **BGE-M3** hỗ trợ đa ngôn ngữ (bao gồm tiếng Việt), mỗi text được chuyển thành một vector có 1024 chiều.
- `chromadb.Client()`: Tạo ChromaDB client lưu trữ trong bộ nhớ (in-memory). Dữ liệu sẽ mất khi tắt chương trình. Nếu muốn lưu lâu dài, dùng `chromadb.PersistentClient(path="./chroma_db")`.
- `client.get_or_create_collection("rag")`: Tạo một "bộ sưu tập" (collection) tên là rag. Nếu đã tồn tại thì dùng lại.
- `collection.add(...)`: Thêm các chunk vào database. Mỗi chunk cần 3 thứ: `ids` (định danh duy nhất), `documents` (text gốc), và `embeddings` (vector số).

💡 Lưu trữ lâu dài 💡

Nếu muốn lưu vector database ra ổ cứng để lần sau không cần embedding lại, thay `chromadb.Client()` bằng `chromadb.PersistentClient(path="./chroma_db")`. ChromaDB sẽ tự động lưu và tải dữ liệu từ thư mục chỉ định.

III.4. Tìm kiếm đoạn liên quan (Retrieve)

Khi có câu hỏi, chúng ta cần tìm những chunk có nội dung liên quan nhất. ChromaDB thực hiện việc này bằng cách so sánh vector của câu hỏi với tất cả vector trong database:

Hàm tìm đoạn liên quan (retrieve)

```

1 def retrieve(query, k=4):
2     """Tìm k đoạn văn bản liên quan nhất với câu hỏi."""
3     res = collection.query(
4         query_embeddings=embed([query]), # Vector hóa câu hỏi
5         n_results=k                       # Số kết quả trả về
6     )
7     return res["documents"][0]
8
9 # Thử tìm kiếm
10 QUERY = "YOLOv10 dùng để làm gì?"
11 for doc in retrieve(QUERY):
12     print(doc[:200])
13     print("-" * 40)

```

Trong đoạn code trên:

- `embed([query])`: Chuyển câu hỏi thành vector, sử dụng cùng model embedding đã dùng cho các chunk.
- `collection.query(query_embeddings=..., n_results=k)`: ChromaDB tự động tính khoảng cách giữa vector câu hỏi và tất cả vector trong database, rồi trả về `k` chunk gần nhất (mặc định là 4).
- `res["documents"][0]`: Lấy danh sách nội dung text gốc của các chunk tìm được.

Tìm kiếm bằng vector hoạt động thế nào? ✂

Khi so sánh 2 vector, ChromaDB đo **khoảng cách** giữa chúng rồi trả về `k` chunk gần nhất với câu hỏi. Mặc định ChromaDB dùng khoảng cách **L2 (Euclid)**: hai vector càng gần nhau thì nội dung càng tương đồng. Nếu muốn dùng **cosine similarity**, ta khai báo khi tạo collection: `get_or_create_collection("rag", metadata={"hnsw:space": "cosine"})`.

III.5. Hỏi đáp với LLM (RAG)

Bước cuối cùng: ghép câu hỏi + các đoạn văn bản tìm được thành một prompt, rồi đưa cho LLM sinh câu trả lời:

Prompt và hàm hỏi đáp RAG

```

1 # Mẫu prompt hướng dẫn LLM cách trả lời
2 PROMPT = """Bạn là trợ lý hỏi đáp. Dùng các đoạn ngữ cảnh dưới đây để trả lời câu hỏi.
3 Nếu ngữ cảnh không có thông tin, hãy nói là bạn không biết, đừng bịa.
4 Trả lời ngắn gọn, chính xác, bằng tiếng Việt.
5
6 Ngữ cảnh:
7 {context}
8
9 Câu hỏi: {question}
10
11 Trả lời: """
12
13 def rag(question, k=4):
14     """Hàm RAG chính: tìm context rồi hỏi LLM."""
15     context = "\n\n".join(retrieve(question, k))
16     resp = ollama.chat(
17         model="vicuna:7b-v1.5-q5_1",
18         messages=[{"role": "user", "content": PROMPT.format(context=context, question=
19                                                             question)}],
20         options={"temperature": 0},
21     )
22     return resp["message"]["content"]
23
24 # Chạy thử
25 print(rag("YOLOv10 là gì?"))

```

Trong đoạn code trên:

- **PROMPT**: Mẫu prompt hướng dẫn LLM cách trả lời. Phần `{context}` sẽ được thay bằng các chunk liên quan, `{question}` thay bằng câu hỏi. Câu "đừng bịa" rất quan trọng: nó ngăn LLM tự sáng tác câu trả lời khi không tìm được thông tin.
- `retrieve(question, k)`: Gọi hàm tìm kiếm ở bước trước, lấy `k` chunk liên quan nhất.
- `'\n\n'.join(...)`: Ghép các chunk thành một chuỗi context duy nhất, ngăn cách bởi 2 dòng trống.
- `ollama.chat(model="vicuna:7b-v1.5-q5_1", ...)`: Gọi mô hình **Vicuna-7B** để sinh câu trả lời.
- `options={"temperature": 0}`: Đặt temperature bằng 0 để LLM trả lời ổn định, không ngẫu nhiên. Mỗi lần hỏi cùng câu hỏi sẽ nhận được cùng câu trả lời.
- `resp["message"]["content"]`: Lấy nội dung text từ phản hồi của LLM.

💡 Temperature là gì? 💡

Temperature kiểm soát mức độ "sáng tạo" của LLM. Giá trị 0 cho câu trả lời chính xác nhất (phù hợp RAG). Giá trị cao (0.7-1.0) cho câu trả lời đa dạng hơn nhưng có thể kém chính xác (phù hợp viết văn sáng tạo).

Như vậy, chúng ta đã hoàn thành một chương trình RAG hoàn chỉnh. Toàn bộ pipeline chỉ gồm 5 hàm/biến chính: `chunk_text()`, `embed()`, `retrieve()`, `PROMPT`, và `rag()`. Mỗi hàm ngắn gọn, dễ đọc, và không phụ thuộc vào framework phức tạp nào.

IV. Xây dựng giao diện với Streamlit

Chương trình RAG ở trên hoạt động tốt, nhưng mỗi lần hỏi chúng ta phải chỉnh code. Để thuận tiện hơn, chúng ta sẽ đóng gói thành ứng dụng web với giao diện thân thiện. Ở đây, chúng ta sử dụng **Streamlit**, một thư viện Python giúp tạo giao diện web nhanh chóng mà không cần biết HTML hay JavaScript.

IV.1. Cài đặt Streamlit

Cài đặt Streamlit

```
1 pip install streamlit
```

IV.2. Tạo file ứng dụng

Tạo file `chatbot_app_native.py` với nội dung sau. Chúng ta sẽ đi qua từng phần của file:

Phần 1: Import và cấu hình

Phần 1: Import và cấu hình

```
1 import streamlit as st
2 import tempfile, os, time
3 import pypdf
4 import chromadb
5 import ollama
6
7 LLM_MODEL = "vicuna:7b-v1.5-q5_1"
8 EMBED_MODEL = "bge-m3"
9
10 PROMPT = """Bạn là trợ lý hỏi đáp. Dùng các đoạn ngữ cảnh dưới đây để trả lời câu hỏi.
11 Nếu ngữ cảnh không có thông tin, hãy nói là bạn không biết, đừng bịa.
12 Trả lời ngắn gọn, chính xác, bằng tiếng Việt.
13
14 Ngữ cảnh: {context}
15
16 Câu hỏi: {question}
17 Trả lời: """
```

Phần này import các thư viện cần thiết và định nghĩa các hằng số. Tên model được tách ra thành biến LLM_MODEL và EMBED_MODEL để dễ thay đổi sau này.

Phần 2: Khởi tạo Session State

Phần 2: Khởi tạo Session State

```
1 for k, v in {"collection": None, "pdf_name": "", "chat_history": []}.items():
2     st.session_state.setdefault(k, v)
```

Session State trong Streamlit

Streamlit chạy lại toàn bộ file Python mỗi khi người dùng tương tác (bấm nút, gõ phím). Nếu không lưu dữ liệu vào `st.session_state`, mọi biến sẽ bị reset về giá trị ban đầu. Ở đây chúng ta lưu 3 thứ: `collection` (vector database đã tạo), `pdf_name` (tên file đang xử lý), và `chat_history` (lịch sử hội thoại).

Đoạn code trên dùng `setdefault` để chỉ khởi tạo giá trị mặc định nếu key chưa tồn tại. Cách viết này gọn hơn so với kiểm tra `if key not in st.session_state` cho từng biến.

Phần 3: Các hàm xử lý (core functions)

Phần 3: Các hàm xử lý chính

```
1 def embed(texts):
2     """Chuyển text thành vector embedding."""
3     return ollama.embed(model=EMBED_MODEL, input=texts)["embeddings"]
4
5 def chunk_text(text, size=1000, overlap=200):
6     """Cắt text thành các chunk nhỏ."""
```

```

7   paras = [p.strip() for p in text.split("\n") if p.strip()]
8   chunks, cur = [], ""
9   for p in paras:
10      if len(cur) + len(p) + 1 <= size:
11         cur += p + "\n"
12      else:
13         if cur:
14            chunks.append(cur.strip())
15            cur = (cur[-overlap:] + p + "\n") if overlap else (p + "\n")
16   if cur.strip():
17      chunks.append(cur.strip())
18   return chunks
19
20 def process_pdf(uploaded_file):
21     """Đọc PDF, cắt nhỏ, tạo embedding và lưu vào ChromaDB."""
22     # Lưu file upload thành file tạm
23     with tempfile.NamedTemporaryFile(delete=False, suffix=".pdf") as tmp:
24         tmp.write(uploaded_file.getvalue())
25         path = tmp.name
26
27     # Đọc nội dung PDF
28     text = "\n".join(p.extract_text() or "" for p in pypdf.PdfReader(path).pages)
29     os.unlink(path) # Xóa file tạm
30
31     # Cắt nhỏ và lưu vào ChromaDB
32     chunks = chunk_text(text)
33     client = chromadb.Client()
34     col = client.get_or_create_collection(f"rag_{int(time.time())}")
35     col.add(
36         ids=[str(i) for i in range(len(chunks))],
37         documents=chunks,
38         embeddings=embed(chunks)
39     )
40     return col, len(chunks)
41
42 def rag(question, collection, k=4):
43     """Hàm RAG: tìm context và hỏi LLM."""
44     res = collection.query(query_embeddings=embed([question]), n_results=k)
45     context = "\n\n".join(res["documents"][0])
46     resp = ollama.chat(
47         model=LLM_MODEL,
48         messages=[{"role": "user", "content": PROMPT.format(context=context, question=
49                                                                question)}],
49         options={"temperature": 0},
50     )
51     return resp["message"]["content"]

```

Hai hàm `embed()` và `chunk_text()` giữ nguyên như phần III. Hàm `rag()` được điều chỉnh để nhận thêm tham số `collection` và gộp luôn bước truy vấn ChromaDB (thay cho việc gọi hàm `retrieve()` riêng). Hàm `process_pdf()` bổ sung thêm logic xử lý file upload từ Streamlit:

- `tempfile.NamedTemporaryFile(...)`: Tạo file tạm trên ổ cứng. File upload từ Streamlit chỉ tồn tại trong bộ nhớ, cần lưu ra file thật để `pypdf` đọc được.

- `os.unlink(path)`: Xóa file tạm sau khi đã đọc xong, tránh chiếm dung lượng ổ cứng.
- `f"rag_{int(time.time())}"`: Tên collection duy nhất theo timestamp, tránh xung đột khi upload nhiều file.

Phần 4: Giao diện người dùng (UI)

Phần 4: Giao diện người dùng

```

1 # Cấu hình trang
2 st.set_page_config(page_title="PDF RAG Chatbot", layout="wide",
3                     initial_sidebar_state="expanded")
4 st.title("PDF RAG Assistant: Native")
5
6 # Sidebar: upload PDF và nút điều khiển
7 with st.sidebar:
8     st.subheader("Upload tài liệu")
9     f = st.file_uploader("Chọn file PDF", type="pdf")
10    if f and st.button("Xử lý PDF", use_container_width=True):
11        with st.spinner("Đang xử lý..."):
12            st.session_state.collection, n = process_pdf(f)
13            st.session_state.pdf_name = f.name
14            st.session_state.chat_history = []
15            st.success(f"{n} chunks")
16            st.info(f" {st.session_state.pdf_name}" if st.session_state.pdf_name
17                  else " Chưa có tài liệu")
18            if st.button("Xóa lịch sử chat", use_container_width=True):
19                st.session_state.chat_history = []
20
21 # Hiển thị lịch sử chat
22 for m in st.session_state.chat_history:
23     with st.chat_message(m["role"]):
24         st.write(m["content"])
25
26 # Ô nhập câu hỏi
27 if st.session_state.collection is None:
28     st.info("Upload và xử lý PDF trước khi chat.")
29     st.chat_input("Nhập câu hỏi...", disabled=True)
30 else:
31     q = st.chat_input("Nhập câu hỏi của bạn...")
32     if q:
33         st.session_state.chat_history.append({"role": "user", "content": q})
34         with st.chat_message("user"):
35             st.write(q)
36         with st.chat_message("assistant"):
37             with st.spinner("Đang suy nghĩ..."):
38                 ans = rag(q, st.session_state.collection)
39                 st.write(ans)
40         st.session_state.chat_history.append({"role": "assistant", "content": ans})

```

Giao diện gồm 3 phần chính:

- **Sidebar (cột trái)**: Chứa nút upload PDF, tên file đang xử lý, và nút xóa lịch sử chat. `st.sidebar` giúp tách phần điều khiển khỏi khu vực chat, giữ giao diện gọn gàng.

- **Lịch sử chat:** Hiển thị toàn bộ tin nhắn trước đó bằng `st.chat_message()`. Mỗi tin nhắn có role ("user" hoặc "assistant") để hiển thị đúng style (người dùng bên phải, chatbot bên trái).
- **Ô nhập câu hỏi:** Dùng `st.chat_input()` tạo ô gõ phía dưới cùng. Khi chưa upload PDF, ô bị tắt (`disabled=True`) để tránh hỏi khi chưa có dữ liệu.

💡 Debug ứng dụng Streamlit 💡

Khi ứng dụng không hoạt động đúng, dùng `st.write(variable)` để in giá trị biến lên giao diện. Ví dụ: `st.write(res)` giúp xem raw output từ ChromaDB trước khi xử lý. Ngoài ra, terminal chạy Streamlit cũng hiển thị lỗi Python chi tiết.

IV.3. Chạy ứng dụng

Mở terminal và chạy:

Chạy ứng dụng trên máy local

```
1 streamlit run chatbot_app_native.py
```

Ứng dụng sẽ mở trên trình duyệt tại địa chỉ `http://localhost:8501`.

Yêu cầu hệ thống

Ứng dụng cần **Ollama** đang chạy trên máy (kiểm tra bằng lệnh `ollama list`). Model `vicuna:7b` cần khoảng 4-5GB RAM. Nếu máy có ít RAM, thay bằng model nhỏ hơn.

Trên **Google Colab**, dùng `localtunnel` để truy cập giao diện web:

Lưu code vào file trên Colab

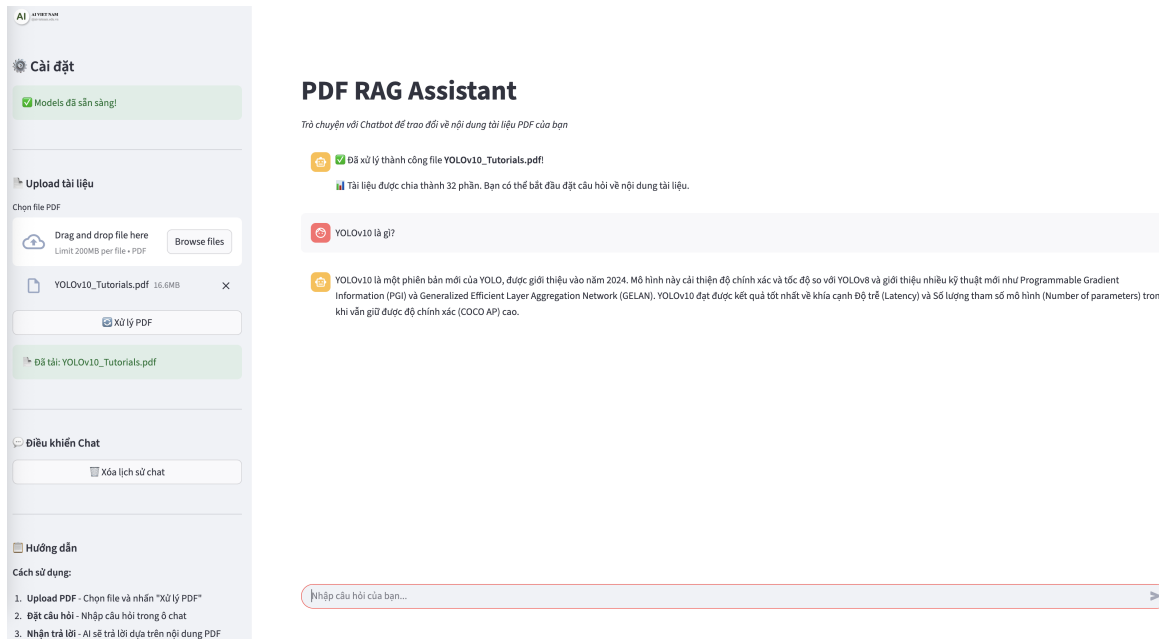
```
1 %%writefile chatbot_app_native.py
2 # (paste toàn bộ code Phần 1-4 ở trên)
```

Chạy ứng dụng với Localtunnel trên Colab

```
1 # Lấy địa chỉ IP (password cho localtunnel)
2 !curl https://loca.lt/mytunnelpassword
3
4 # Chạy ứng dụng
5 !streamlit run chatbot_app_native.py & npx localtunnel --port 8501
```

💡 Sử dụng Localtunnel trên Colab 💡

Khi bạn click vào link của localtunnel (có dạng `https://....loca.lt`), trang web sẽ yêu cầu nhập "Endpoint IP". Hãy copy dãy số IP hiện ra từ lệnh `!curl https://loca.lt/mytunnelpassword` và dán vào đó để tiếp tục.



Hình 3: Giao diện Chatbot với lịch sử hội thoại

Quy trình sử dụng:

- Upload PDF:** Chọn file PDF ở sidebar và nhấn "Xử lý PDF". Hệ thống sẽ đọc, cắt nhỏ, và tạo vector database.
- Hỏi đáp:** Nhập câu hỏi ở ô chat phía dưới. Chatbot sẽ tìm đoạn liên quan rồi trả lời dựa trên nội dung tài liệu.
- Xem lịch sử:** Lịch sử hội thoại được lưu trữ suốt phiên làm việc. Nhấn "Xóa lịch sử chat" để bắt đầu hội thoại mới.

Tổng kết

Trong bài viết này, chúng ta đã xây dựng thành công một hệ thống RAG Chatbot bằng Python thuần, không phụ thuộc vào framework phức tạp:

- Hiểu pipeline RAG:** Nắm vững luồng xử lý cơ bản (Chunking → Embedding → Vector Database → Retrieval → LLM Generation).
- Code native với 3 thư viện:** Dùng `pypdf` (đọc PDF), `chromadb` (vector database), và `ollama` (embedding + LLM).
- Tự viết hàm chunking:** Hiểu rõ cơ chế cắt văn bản bằng cách tự viết hàm `chunk_text()` (~10 dòng).
- Đóng gói bằng Streamlit:** Xây dựng giao diện chatbot hoàn chỉnh với lịch sử hội thoại.

Gợi ý cải tiến

Project xây dựng chatbot hỏi đáp đơn giản thông qua kiến trúc RAG. Để tiếp tục cải tiến hệ thống chuyên sâu hơn cho tiếng Việt, dưới đây là một số hướng gợi ý:

- Nâng cấp mô hình Embedding (Vector hóa):** Model `bge-m3` trong bài là lựa chọn tốt, nhưng bạn có thể thử thêm:
 - `nomic-embed-text`: Nhẹ hơn, tốc độ nhanh, phù hợp máy cấu hình thấp. Cài bằng `ollama pull nomic-embed-text` rồi đổi `EMBED_MODEL = "nomic-embed-text"`.
 - Nếu dùng Python thuần (không qua Ollama), thử `sentence-transformers` với model `BAAI/bge-m3` hoặc `intfloat/multilingual-e5-large`, là model embedding đa ngôn ngữ mạnh trên bảng xếp hạng MTEB và xử lý tiếng Việt tốt.
- Nâng cấp mô hình LLM (Sinh văn bản):** Tất cả model Ollama đều đổi được chỉ bằng cách thay biến `LLM_MODEL`. Hãy thử:
 - `qwen2.5:3b`: Nhẹ hơn, chỉ cần ~2GB RAM. Phù hợp máy cấu hình thấp.
 - `llama3.2:latest`: Model của Meta, mạnh mẽ và đa năng.
 - `gemma2:9b`: Model của Google, xử lý tiếng Việt khá tốt.

💡 Lời khuyên 💡

Bạn chỉ cần chạy `ollama pull <tên_model>` rồi đổi biến `LLM_MODEL` trong code. Không cần thay đổi gì khác. Hãy thử nhiều model để tìm model phù hợp nhất với tài liệu của bạn!

- Lưu trữ lâu dài:** Thay `chromadb.Client()` bằng `chromadb.PersistentClient()` để vector database lưu ra ổ cứng. Lần sau khởi chạy, ứng dụng dùng lại dữ liệu đã có mà không cần xử lý PDF lại.
- Kết hợp phương pháp tìm kiếm:** Sử dụng **Hybrid Search** bằng cách kết hợp thuật toán BM25 (tìm theo từ khóa) với Cosine Similarity (tìm theo ngữ nghĩa) để tăng độ chính xác truy tìm tài liệu.

V. Câu hỏi trắc nghiệm

1. Xét hàm `chunk_text()` trong bài với `size=1000`, `overlap=200`. Giả sử văn bản gốc có 5 đoạn (paragraph) với chiều dài lần lượt là: 400, 500, 300, 600, 200 ký tự (chưa tính ký tự `\n` mà hàm thêm vào khi ghép). Hàm xử lý bằng cách duyệt từng đoạn và gộp vào chunk hiện tại cho tới khi vượt quá `size`.

Hỏi: Sau khi chạy `chunk_text()`, có bao nhiêu chunk được tạo ra?

- A. 2
 - B. 3
 - C. 4
 - D. 5
2. Trong hàm `chunk_text()`, khi chunk hiện tại (`cur`) đã đầy và bắt đầu chunk mới, dòng code thực hiện `overlap` là:

```
1 cur = (cur[-overlap:] + p + "\n") if overlap else (p + "\n")
```

Giả sử `cur` hiện tại là "ABCDEFGHGIJ" (10 ký tự), `overlap=4`, và đoạn mới `p` = "XYZ". Hỏi giá trị của `cur` sau dòng code trên là gì?

- A. "GHIJXYZ\n"
 - B. "ABCDEFGHGIJXYZ\n"
 - C. "XYZGHIJ\n"
 - D. "GHIJ\nXYZ\n"
3. Xét đoạn code đọc file PDF:

```
1 full_text = "\n".join(page.extract_text() or "" for page in reader.pages)
```

Phần `or ""` trong đoạn code trên có vai trò gì?

- A. Bắt buộc phải có, nếu thiếu thì mọi file PDF đều gây lỗi khi chạy.
 - B. Là cách phòng thủ: khi một trang không trích được text (giá trị rỗng hoặc `None`), `or ""` thay bằng chuỗi rỗng để `"\n".join()` luôn ghép được, tránh lỗi.
 - C. Giúp tăng tốc độ trích xuất text từ các trang.
 - D. Tự động xóa các trang không có text ra khỏi tài liệu.
4. Xét hàm `embed()` và cách gọi nó trong hàm `retrieve()`:

```

1 def embed(texts):
2     return ollama.embed(model="bge-m3", input=texts)["embeddings"]
3
4 def retrieve(query, k=4):
5     res = collection.query(
6         query_embeddings=embed([query]), # Gọi embed([query])
7         n_results=k
8     )
9     return res["documents"][0]

```

Trong `retrieve()`, vì sao nên truyền `embed([query])` (một list) thay vì gọi trực tiếp `embed(query)`?

- A. Vì `embed` là API xử lý theo lô: `input` là danh sách chuỗi và khoá `"embeddings"` trả về danh sách vector. Bọc `[query]` để kết quả là danh sách vector, khớp định dạng mà `collection.query(query_embeddings=...)` mong đợi.
- B. Vì nếu truyền chuỗi đơn, Ollama sẽ tách từng ký tự thành một đoạn text riêng biệt.
- C. Vì cú pháp `[query]` tạo một bản sao của chuỗi để tránh framework thay đổi biến gốc.
- D. Vì `query_embeddings` bắt buộc phải là một số thực một chiều.

5. Trong ứng dụng Streamlit (phần IV), hàm `process_pdf()` tạo collection với tên:

```

1 col = client.get_or_create_collection(f"rag_{int(time.time())}")

```

Trong khi ở phần III (chương trình console), tên collection cố định là `"rag"`. Tại sao phiên bản Streamlit dùng timestamp?

- A. Vì ứng dụng Streamlit có tốc độ xử lý nhanh hơn, do đó việc sử dụng timestamp giúp phân biệt rõ ràng các request đến từ nhiều người dùng tại cùng một thời điểm.
- B. Vì mỗi lần tải lên file PDF mới, hệ thống cần thiết lập một collection độc lập nhằm tránh tình trạng dữ liệu của file cũ bị trộn lẫn vào kết quả tìm kiếm của thao tác mới.
- C. Vì đối tượng `chromadb.Client()` được khởi tạo trong môi trường Streamlit có quy định tên collection bắt buộc phải chứa ký tự số, không chấp nhận chuỗi chỉ chứa chữ cái thuần.
- D. Vì timestamp đóng vai trò như một metadata quan trọng giúp ChromaDB tự động sắp xếp các collection theo thứ tự tạo ra, từ đó hỗ trợ tốt cho tính năng rollback hệ thống.

6. Xét prompt template trong hàm `rag()`:

```

1 PROMPT = """Bạn là trợ lý hỏi đáp. Dùng các đoạn ngữ cảnh dưới đây để trả lời câu hỏi
2             i.
3 Nếu ngữ cảnh không có thông tin, hãy nói là bạn không biết, đừng bịa.
4 Trả lời ngắn gọn, chính xác, bằng tiếng Việt.
5 Ngữ cảnh:

```

```

6 {context}
7
8 Câu hỏi: {question}
9
10 Trả lời: ""

```

Nếu bỏ dòng "Nếu ngữ cảnh không có thông tin, hãy nói là bạn không biết, đừng bịa" khỏi prompt, hậu quả nghiêm trọng nhất là gì?

- A. Mô hình ngôn ngữ sẽ đưa ra câu trả lời chậm hơn đáng kể do hệ thống phải tự phân tích và suy luận thêm nhiều thông tin phức tạp vốn không có sẵn trong dữ liệu bối cảnh.
 - B. Mô hình ngôn ngữ có xu hướng tự tạo ra câu trả lời dựa trên kiến thức đã được huấn luyện thay vì dựa vào tài liệu, dễ dẫn đến hiện tượng "ảo giác" thông tin (hallucination).
 - C. Mô hình ngôn ngữ sẽ liên tục báo lỗi hoặc hệ thống từ chối trả lời mọi câu hỏi của người dùng tiếp theo do không nhận được đủ hướng dẫn cần thiết trong prompt template.
 - D. Mô hình ngôn ngữ sẽ tự động chuyển sang cấu trúc trả lời bằng tiếng Anh hoặc ngôn ngữ mặc định khác thay vì tiếng Việt do thiết lập prompt bị thiếu ràng buộc ngôn ngữ.
7. Trong bài, model embedding **bge-m3** tạo vector 1024 chiều cho mỗi đoạn text. Xét tình huống: tài liệu PDF có 200 chunk, mỗi chunk được embed thành vector 1024 chiều. Khi user hỏi 1 câu hỏi, ChromaDB cần so sánh vector câu hỏi với bao nhiêu vector trong database để tìm ra **k=4** chunk liên quan nhất?
- A. Chỉ tiến hành so sánh với 4 vector tương ứng với 4 chunk đầu tiên trong cơ sở dữ liệu đã được hệ thống sắp xếp hoặc trích xuất từ trước.
 - B. Tiến hành so sánh câu hỏi với tất cả 200 vector đang có trong cơ sở dữ liệu ChromaDB, sau đó tính toán để chọn ra top-4 vector có độ tương đồng cao nhất.
 - C. Hệ thống sẽ thực hiện theo hướng so sánh từng chiều (trong tổng số 1024 chiều) của vector câu hỏi gốc với các vector có trong toàn bộ cơ sở dữ liệu.
 - D. Hệ thống sẽ phải chạy tổng cộng 204,800 phép tính vector độc lập (200 x 1024) trước khi đưa ra được kết quả cuối cùng chứa 4 document tương đồng lớn nhất.

8. Xét Session State trong ứng dụng Streamlit:

```

1 for k, v in {"collection": None, "pdf_name": "", "chat_history": []}.items():
2     st.session_state.setdefault(k, v)

```

Một sinh viên sửa thành:

```

1 st.session_state["collection"] = None
2 st.session_state["pdf_name"] = ""
3 st.session_state["chat_history"] = []

```

Hai cách viết này có **hành vi khác nhau** khi nào?

- A. Về cơ bản không có điểm khác biệt đặc biệt: cả hai phương pháp đều đảm nhiệm vai trò khởi tạo các giá trị vòng lặp mặc định cho phiên hoạt động (session state).
 - B. Khi người dùng đang tương tác: lệnh `setdefault` giúp bảo toàn dữ liệu hiện có, trong khi lệnh gán trực tiếp sẽ liên tục ghi đè, gây ra lỗi xóa sạch mọi dữ liệu sau mỗi lần Streamlit tải lại (rerun).
 - C. Lệnh gán trực tiếp có tốc độ thực thi nhanh hơn rõ rệt vì nó bỏ qua hoàn toàn bước kiểm tra xem từ khóa (key) đã tồn tại trong trạng thái phiên máy chủ hay chưa.
 - D. Lệnh `setdefault` chỉ tương thích với cấu trúc dữ liệu dạng dictionary, trong khi phương thức gán trực tiếp còn lại có thể hoạt động ổn định với mọi kiểu cấu trúc dữ liệu khác nhau.
9. Giả sử tài liệu PDF chứa một bảng so sánh 3 model YOLO (v5, v8, v10) với các cột: FPS, mAP, số tham số. Bảng nằm trọn trong 1 trang PDF. Khi dùng `pypdf` với `page.extract_text()`, kết quả extract có thể gặp vấn đề gì?
- A. Thư viện `pypdf` hiện chưa được trang bị tính năng đọc trực tiếp file PDF có chứa bảng biểu, do đó hàm sẽ bỏ qua phần này và trả về một chuỗi ký tự rỗng.
 - B. Cấu trúc bảng (hàng/cột) có khả năng bị phá vỡ hoàn toàn do `extract_text()` chỉ tập trung trích xuất văn bản thuần (plain text) mà không duy trì giới hạn hiển thị của layout bảng.
 - C. Thư viện `pypdf` sẽ tự động phát hiện và chuyển đổi định dạng của toàn bộ bảng sang cấu trúc JSON tiêu chuẩn, yêu cầu lập trình viên phải lập trình một hàm parse riêng biệt.
 - D. Toàn bộ bảng biểu cùng dữ liệu sẽ được phần mềm giữ nguyên vẹn định dạng hiển thị gốc vì bản thân hệ sinh thái `pypdf` đã sở hữu sẵn công cụ hỗ trợ cấu trúc bảng phức tạp.
10. Một sinh viên muốn cải tiến hệ thống RAG bằng cách thêm **bộ nhớ hội thoại** (conversation memory). Cách tiếp cận nào sau đây là **đúng kỹ thuật** và **khả thi** nhất?
- A. Tiến hành lưu toàn bộ biến `chat_history` vào cơ sở dữ liệu vector ChromaDB cùng với collection "rag", phương pháp này giúp ChromaDB có đủ dữ liệu tìm kiếm toàn thời gian.
 - B. Sửa đổi hàm `rag()` để cấu hình truyền thông số `chat_history` thông qua chuỗi `messages` cho api `ollama.chat()`, giúp mô hình ngôn ngữ lớn tiếp nhận đầy đủ toàn bộ ngữ cảnh hội thoại trước đó.
 - C. Cấu hình thêm một mô hình ngôn ngữ thứ hai chuyên biệt đảm nhiệm việc xử lý và duy trì bộ nhớ người dùng, trong khi mô hình ngôn ngữ thứ nhất chỉ tập trung vào việc tạo nội dung trả lời.
 - D. Tiến hành nâng mức kích thước `chunk_size` trong hệ thống lên giới hạn 10,000 ký tự để mỗi chunk có thể lưu trữ đa dạng thông tin hơn, từ đó làm giảm đi đáng kể nhu cầu duy trì bộ nhớ lịch sử.

Phụ lục

- Hint:** Các file code gợi ý và dữ liệu nếu có được lưu trong thư mục có thể được tải [tại đây](#).
- Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải [tại đây](#) (**Lưu ý:** Sáng thứ 3 khi hết deadline phần project, ad mới copy các nội dung bài giải nêu trên vào đường dẫn).
- Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 giờ.

AIO_QAs-Verified

🔒 Nhóm Riêng tư · 1,4K thành viên



Hình 4: Hình ảnh group facebook AIO Q&A

4. Rubric:

Phần	Kiến Thức	Đánh Giá
1.	- Hiểu pipeline RAG: Chunking → Embedding → Vector Database → Retrieval → LLM Generation - Nắm vai trò của <code>chunk_text</code>, <code>embed</code>, <code>retrieve</code>, <code>PROMPT</code> và <code>rag</code> - Hiểu cơ chế chunk/overlap, embedding và tìm kiếm theo khoảng cách vector trong ChromaDB	- Cài đặt được pipeline RAG bằng Python thuần với <code>pypdf</code>, <code>chromadb</code>, <code>ollama</code> - Giải thích đúng vì sao cần chunking, overlap và prompt "đừng bịa" - Hiểu ý nghĩa các tham số <code>n_results</code>, <code>temperature</code>
2.	- Hiểu cách đóng gói pipeline RAG thành ứng dụng Streamlit - Nắm vai trò của <code>st.session_state</code>, <code>st.file_uploader</code>, <code>st.chat_input</code> và <code>st.chat_message</code> - Hiểu vì sao cần file tạm khi đọc PDF upload	- Xây dựng được giao diện chatbot có upload PDF, hỏi đáp và lưu lịch sử hội thoại - Giải thích đúng luồng xử lý từ lúc upload đến khi trả lời - Biết cách chạy ứng dụng trên máy local và Google Colab

- Hết -